

A Parallel Implementation of the Fast Multipole Method

CSCI-GA 3033: Multicore Processors

Peter Scott Clint Goodwin

April 2026

Abstract In this paper we describe and implement a parallel version of the Fast Multipole Method (FMM). The FMM is an algorithm for evaluating potential at a large number of points in a force field. In particular, the algorithm allows computation of potential at N points up to a given precision ϵ with $\mathcal{O}(N)$ operations, an improvement over direct computation with $\mathcal{O}(N^2)$ operations. Numerical results demonstrate a high degree of parallelization and performance increase from optimal parameter selection.

1 Introduction

The FMM is a numerical algorithm used to estimate pairwise interactions between a collection of N particles in a grid. The exact problem can be described mathematically as evaluating

$$p_i = \sum_{j=1}^N K(x_i, x_j) q_j \quad i = 1, \dots, N$$

where N is the number of particles, x_i is the location of the i -th particle in the complex plane, q_i is the charge at x_i , and K is the interaction kernel. In this implementation, we are concerned with calculating the electric potential in a two dimensional plane, thus we will use the Laplace kernel:

$$K(x_i, x_j) = \begin{cases} -\frac{\log |x_i - x_j|}{4\pi} & x_i \neq x_j \\ 0 & x_i = x_j \end{cases}$$

and consider q_i as integer-valued charges. We note that the FMM can be modified to solve the above problem in three dimensions and using different kernels (e.g., Helmholtz, Yukawa) but we will not go into detail on those implementations here.

The FMM is able to estimate the potentials at all of the points up to a given relative error ϵ using $\mathcal{O}(N)$ operations, with a constant that scales with $\log(1/\epsilon)$. It achieves this by creating a tree structure that represents the grid space: the algorithm recursively partitions squares in the grid into four quadrants, calculates estimates for the charge distributions of the tree's leaf nodes, and – using these charge distributions – derives estimates for nearby nodes in the tree. Charge distributions are encoded as sequences of complex numbers of length P called expansions.

The algorithm can be divided into four consecutive procedures that, for the most part, must be completed sequentially:

1. **Tree Construction** Starting with a box containing the entire domain, recursively partition the grid into four child boxes of equal size. Once the original domain has been completely partitioned, every particle will be contained in exactly one of the lowest level boxes (“leaf nodes”).
2. **Multipole Expansions** Starting at the leaf nodes, compute multipole expansions using the *outgoing from sources* operator with source charges in a single box as input. Next, compute higher-level multipole expansions using the *outgoing from outgoing* operator with multipole expansions of child boxes as input.

3. **Local Expansions** Starting from the second level, compute local expansions using the sum of *incoming from outgoing* and *incoming from incoming* operators with outgoing multipole expansions of well-separated boxes and incoming expansions of parent boxes as input. This represents the contributions from well-separated particles via the boxes they are contained in: particles within boxes in the “interaction list” of a node are captured via their outgoing multipole expansion, otherwise those already well-separated from its parent are inherited via their parent’s incoming local expansion.
4. **Potential Evaluation** For each point in the grid, compute the point’s potential using the sum of the *target from incoming* operator, representing contributions from well-separated particles, and direct evaluation of near-field potential as a result of charges in the box itself and its neighboring boxes.

Martinsson [4] provides an in-depth description of the algorithm and formulas for the expansion operators using the Laplace kernel. We note that the expansion operators are complex-valued linear operators dependent on dimensions of expansions and locations of boxes.

2 Literature Survey

The FMM was first developed and introduced by Greengard and Rokhlin [3] in 1987 for quickly computing pairwise interactions in electric or gravitational fields. This type of problem uses the Laplace kernel. The algorithm was extended by Engheta [1] to solve acoustic scattering problems using the Helmholtz kernel and has since been applied to a wide variety of particle simulation problems.

Efforts have been made to quantify the extent of parallelism [2] and optimize the implementation of the algorithm for shared memory machines [5]. In this implementation, we attempt to achieve major performance gains from parallelism while maintaining clarity in code and simplicity where possible.

3 Proposed Idea

Our proposed parallel implementation of the FMM exploits the inherent hierarchical structure of the algorithm while also incorporating optimizations such as sequential truncation and first-touch memory locality to support non-uniform memory access (NUMA) architectures. Further, our primary data structures – the FMM tree and the particle data – are designed to scale efficiently with increasing parallelism.

3.1 Parallelism from hierarchical structure

The key to our implementation is the appropriate placement of parallel regions in sections of the algorithm that support parallelism:

1. **Tree Construction** Recursively partitioning the particle region into four quadrants until reaching a desired level of depth can be completely parallelized: at a given level of depth, each quadrant is fully independent of other quadrants.
2. **Multipole Expansions** In the calculation of multipole expansions during the algorithm’s “upward pass,” initial computation of the *outgoing from sources* can be fully parallelized across the tree’s leaf nodes since each leaf node’s outgoing multipole expansion is independent of other leaf nodes. After outgoing expansions are computed at the leaf level, each subsequent level of tree can compute its *outgoing from outgoing* expansion (using child node expansions as input) in parallel. We note that there is an implicit barrier between each level: a level must complete its multipole expansion calculation before proceeding to the next level since the subsequent level depends on data from the current level.
3. **Local Expansions** Calculation of local expansions in the “downward pass” can be similarly parallelized: each node in a given level can compute its *incoming from incoming* and *incoming from outgoing* local expansion contributions independent of other nodes in that level, but each level must be complete before proceeding to the next. These calculations require each node has access to their parent’s incoming local expansion, and thus a barrier is placed between each level.

4. **Potential Evaluation** Potentials be computed in parallel across leaf nodes: each particle calculates its potential using the local expansion of the leaf node containing it and adds near-field contributions from particles in neighboring nodes via direct calculation. Note the per-particle direct calculation of near-field contributions is computed sequentially because the possible gains in performance do not justify the cost of parallelism overhead at this level of granularity (particles can be in the millions).

The parallelism described in the above sections represents the vast majority of modifications to the sequential version of the FMM algorithm to achieve significant parallelism speedup. We note that these changes require a sophisticated understanding of the FMM algorithm’s structure but – assuming the sequential version is implemented thoughtfully – do not require significant modifications to the underlying code; much of the above can be achieved with well-placed `#pragma omp parallel for` statements.

3.2 Additional optimizations

Beyond the parallelism inherent to the hierarchical structure of the FMM, our implementation adds additional optimizations to further support scalability.

At a certain level of tree depth during the upward and downward passes, performance gains from parallelism will not justify the additional overhead cost of thread management. For example, at the second level of the tree, there are only $4^2 = 16$ nodes; this is sufficiently small that the required memory to support node-level operations could easily fit into a core’s L1 or L2 cache, and as a result sequential calculation of these results would likely be faster. Our implementation takes this into account by truncating the parallelism of the per-level operations at a given depth using a threshold level for parallelism, below which the node-level operations run sequentially.

Our implementation also includes modifications to utilize first-touch memory locality to support NUMA architectures. Tree construction performs an in-place sort of the input particle data, resulting in a list of tree nodes that have index ranges pointing to a corresponding list of ordered particle data. By mirroring the computation access pattern for the tree nodes, our first-touch memory locality initialization places multipole and local expansion data in memory proximate to the threads responsible for computing the expansions for that node. Furthermore, the particle data is copied from its original memory location – specified randomly at input time during the sequential read of data – to a memory location proximate to the thread that will access that particle data during leaf node operations. These optimizations ensure that, for a sufficiently large problem running with threads that spill beyond a single NUMA node, data storage will ensure memory locality across all threads performing computations.

3.3 Data structure implementation

The choice of data structure design and implementation for the particle data and quad tree in the FMM is important as it relates to scaling performance with increasing parallelism. In our implementation, the particle data is stored as an array of `structs` that is sorted in-place during tree construction, with the resulting tree nodes pointing to an index range of particles in the sorted array. The tree is represented as an array of “node” `structs` such that each node has a level, spatial midpoint, particle index range, and pointers to local and multipole expansions. Importantly, the memory for the local and multipole expansions of tree nodes is allocated as a single contiguous block of memory at startup time: this ensures that neighboring nodes will have their expansion data stored in neighboring sections of memory, increasing the likelihood of cache hits when performing calculations using expansion data on subsequent nodes.

This data structure implementation allows us to maximize the benefits of parallelism by thoughtfully exploiting the memory layout structure of the underlying arrays containing particle and tree node data. Alternative choices, such as a linked list for the tree nodes, may have been easier to implement but would suffer from poor memory locality and thus scale poorly with increasing parallelism.

4 Experimental Setup

Our project submission contains four programs that can be compiled via the Makefile:

1. **fmm** – executable of the parallelized FMM implementation. This runs the FMM algorithm on the input particle data file and writes the resulting particle data (with computed potentials) to the specified location, with required parameters such as the desired depth of the tree, the number of expansions (multipole and local) to compute, the count of particles, and how many threads to run. Intended usage: `./fmm <input_file> <output_file> <num_levels> <P> <is_parallel> <num_threads> [first_touch]`
2. **direct** – executable containing implementations of a direct potential calculation algorithm (both sequential and parallel options) for validating FMM output and timing comparison. Intended usage: `./direct <input_file> <output_file> <num_particles> <is_parallel> <num_threads>`
3. **compare** – executable to evaluate outputs created by **fmm** and **direct** for validation. Intended usage: `./compare <output_file_1> <output_file_2> <num_particles>`
4. **data_writer** – executable to create test particles data for input to **fmm** and **direct**. Intended usage: `./data_writer <num_particles> <output_file>`

For the experiments presented in the next section, we used NYU CIMS **crunchy** machines (specifically, **crunchy2** and **crunchy3**) with 64 total cores over 4 sockets and 8 NUMA nodes, with a 16 KiB per-core L1 data cache, 64 KiB per 2-core module shared L1 instruction cache, 2 MiB per 2-core module shared L2 cache, and 6 MiB per 8-core die shared L3 cache.

To perform the experiments, we created input data files of different particle sizes using **data_writer** before then testing **fmm** and **direct** using various parameter values (e.g., tree depth, number of threads, etc.). We then used **compare** to calculate the relative error ϵ between two outputs, typically using the output of the sequential direct computation as a source to validate against.

Recall the relationship between relative error ϵ and number of digits of accuracy:

$$\# \text{ of digits of accuracy} = -\log_{10} \epsilon$$

and 32-bit single precision floating point arithmetic gives roughly 7 significant digits of precision. As a result, most of our tests would aim to vary input parameters while keeping a fixed ϵ value of approximately 10^{-7} resulting from **compare** to match 32-bit single precision (our implementation uses 32-bit **float** values throughout).

Timing of the programs are measured within the C code using the OpenMP `omp_get_wtime()` function to measure the elapsed wall clock time in seconds for blocks of source code. The resulting elapsed time of the computation is printed out by the programs **fmm** and **direct** for comparison purposes.

5 Experiments & Analysis

We performed four experiments to: (i) demonstrate the performance improvement of the parallel FMM implementation versus sequential FMM implementation and direct potential evaluation methods; (ii) illustrate relative speedup of the parallel FMM implementation for various problem sizes; (iii) validate the optimal choice of a key parameter in parallel FMM performance (B , the average particle density per leaf node); and (iv) evaluate the effectiveness of our first-touch memory locality implementation.

5.1 Parallel FMM versus other methods

The first experiment demonstrated speedup achieved by the FMM and further speedup achieved by the parallel implementation. The parallel tests used 16 threads. The FMM tests used $P = 8$ and the number of levels l to achieve the theoretically optimal B for each size of problem (the theoretically optimal B is roughly 32, shown experimentally in 5.3).

		$N = 2,048$	$N = 8,192$	$N = 32,768$	$N = 131,072$
Direct	Serial	2.370 s	197.105 s	-	-
	Parallel	0.338 s	5.718 s	100.114 s	650.621 s
		$P = 8$	$P = 8$	$P = 8$	$P = 8$
		$l = 3$	$l = 4$	$l = 5$	$l = 6$
FMM	Serial	0.296 s	2.510 s	6.346 s	15.412 s
	Parallel	0.126 s	0.653 s	1.983 s	5.025 s

Table 1: Overall performance of different algorithms for a given problem size up to a fixed relative error ϵ .

For both serial and parallel implementations, FMM scales at approximately $\mathcal{O}(N)$, whereas the direct evaluation scales at approximately $\mathcal{O}(N^2)$. With very large problem sizes, the linear scaling drastically outperforms the direct calculation, as we expect. With the calculated ϵ matching machine epsilon for 32-bit single precision, the estimate calculated by FMM is equally as precise as the direct evaluation. The parallel implementation did not increase speed as much as expected; this is likely due to competing resources on the **crunchy** machines, which increased CPU costs of thread handling, and some inherently sequential parts of the FMM algorithm.

5.2 Speedup for parallel FMM

The second experiment tested the speedup of the parallel FMM as thread count increased. Again, the tests use $P = 8$ and number of levels l such that B is near optimal. Note that the first row contains raw time values and not speedup.

		$N = 2,048$	$N = 8,192$	$N = 32,768$	$N = 131,072$
		$P = 8$	$P = 8$	$P = 8$	$P = 8$
threads		$l = 3$	$l = 4$	$l = 5$	$l = 6$
1	(0.142 s)	(0.626 s)	(3.938 s)	(10.836 s)	
2	1.667	1.883	1.645	1.944	
4	1.332	2.409	2.960	3.682	
8	0.317	3.217	3.463	6.644	
16	0.113	3.816	5.566	7.903	

Table 2: Speedup achieved for parallel FMM relative to sequential implementation for a given problem size.

The first observation is that the speedup increases nearly uniformly as the problem size increases for each thread value. Additionally, the efficiency of marginal threads decreases as the number of threads increases to 16 for all problem sizes. It is likely that we would observe greater efficiency in high thread counts for larger problem sizes, since the cost of maintaining 16 threads can be relatively significant to the performance of a parallel program that is running on the scale of one second whereas that cost would be more effectively amortized over a larger problem size.

5.3 Optimal parameter choice in parallel FMM

The third experiment aimed to experimentally verify the claim in Greengard [2] that the optimal density B is roughly 30 points per box for single precision accuracy. For each problem size, the value of P is set such that the FMM computes potentials to a relative precision of 10^{-6} . For each test run, l is modified to achieve the necessary value of B . For example, the run with $(B = 32, N = 8,192)$ uses $l = 4$ since the fourth level contains 256 boxes.

	$N = 8,192$	$N = 32,768$	$N = 131,072$
B	$P = 7$	$P = 7$	$P = 6$
0.125	21.920 s	39.933 s	91.410 s
0.5	5.381 s	17.803 s	7.0179 s
2	3.020 s	4.490 s	1.686 s
8	0.499 s	1.988 s	1.360 s
$B_{opt} \approx 32$	0.131 s	0.559 s	1.021 s
128	0.171 s	2.407 s	7.505 s
512	1.648 s	10.092 s	9.544 s
2048	19.191 s	57.202 s	170.288 s

Table 3: Maximum speedup achieved for parallel FMM on a given problem size for various choices of B .

The table shows that all problem sizes achieve optimal performance with $B = 32$, which experimentally verifies the claim that $B_{opt} \approx 30$. The optimal selection of B balances the number of direct potential evaluations and the number of leaf box expansion computations. As B increases, the algorithm uses fewer expansions to estimate the potential distribution within regions, which results in more direct potential evaluations. In the degenerate case of $B = N$, the FMM is equivalent to the direct calculation – the algorithm slows to $\mathcal{O}(N^2)$. On the other hand, as B decreases, the algorithm performs a full expansion to estimate the potential for fewer particles. With $B = 0.125$, a high proportion of particles are alone in their leaf box – and some leaf boxes are completely empty; the work done to calculate the expansions representing the charge distribution in empty boxes is wasted.

5.4 First-touch memory locality

In this experiment, we aimed to test the performance of the parallel FMM implementation using first-touch memory locality on NUMA architectures versus not. We chose a sufficiently large problem size – 1,000,000 particles – such that the particles array could not fit easily in cache; this forced the effects of non-uniform memory distribution to be felt. Furthermore, we chose a depth $l = 8$ (corresponding to an average density $B \approx 15$, as close as possible to optimal for this number of particles) and expansion size $P = 10$ such that we would efficiently compute potentials for each particle up to 32-bit single precision $\epsilon \approx 10^{-7}$.

Threads	No First-Touch (s)	First-Touch (s)
4	31.013	34.557
8	16.911	18.878
16	10.632	9.440
32	6.280	5.738

Table 4: FMM particle potential evaluation time for 1,000,000 particles ($l = 8$, $P = 10$) with and without first-touch memory placement across thread counts. First-touch improves performance once threads span multiple NUMA nodes (≥ 16 on **crunchy2**), if particles array exceeds per-node L3 cache size (6 MB).

Testing this problem size using 4, 8, 16, and 32 threads shows the implementation using first-touch data initialization begins to outperform the implementation without first-touch at a thread count of 16; before then, performance gains from improved memory locality do not justify the overhead cost of duplicate data initialization (copying particles array to a new memory location and re-initializing expansion memory).

These results match our intuition regarding NUMA architectures: with the testing environment of CIMS **crunchy** machines (a NUMA node has 8 cores), we would expect that with 8 or fewer threads, all of the work could fit entirely on a single NUMA node. First-touch memory placement would add no benefit in this case and only incur additional cost, leading to faster performance by the implementation without first-touch memory placement. Beyond 8 threads, performance of parallel FMM continues to improve and work begins to spill over beyond a single NUMA node. We expect parallel FMM using first-touch memory placement with ≥ 16 threads to consistently outperform the implementation without first-touch, matching our results.

6 Conclusions

As a standalone algorithm, the FMM is an effective tool: scaling at $\mathcal{O}(N)$ complexity versus $\mathcal{O}(N^2)$ complexity of the direct pair-wise force potential calculation makes problems of a certain size tractable that would otherwise not be. The work in this paper additionally shows the strength of the FMM as it relates to implementing a parallel version of the algorithm; this parallel implementation leverages the FMM's inherent structure to scale effectively as parallelism grows, thus enabling simulations of large systems that would have been previously impossible (e.g., simulating interactions between galaxies with millions of stars, fine discretizations of time-evolved physical systems, etc.).

The work of implementing the parallel FMM and conducting the numerical experiments yields the following lessons:

1. For a given problem, the choice of algorithm dictates how much speedup one can expect. Choosing an algorithm that inherently parallelizes well can lead to a significant speedup with relatively few modifications needed, resulting in an implementation that exceeds the performance of a naive algorithm with a lot of fine-tuning.
2. For a given algorithm, choice of parameters has a large effect on performance and speedup. In the case of FMM, the optimal density of particles per leaf node B_{opt} has a dramatic impact on performance. In some cases, the wrong choice can result in a program that is outperformed by a naive approach which uses thoughtful parallelization optimizations.
3. Memory locality requires thoughtful choices about data structure design and implementation. Optimal data structures can be difficult to implement and advanced optimizations like first-touch memory placement can incur a cost that is not justified by performance gains. Careful consideration is needed for choosing the right implementation and deciding when an optimization will justify its additional cost.

References

- [1] N. Engheta, W.D. Murphy, V. Rokhlin, and M.S. Vassiliou. The fast multipole method (fmm) for electromagnetic scattering problems. *IEEE Transactions on Antennas and Propagation*, 40(6):634–641, 1992.
- [2] L. Greengard and W.D. Gropp. A parallel version of the fast multipole method. *Computers Mathematics with Applications*, 20(7):63–71, 1990.
- [3] L Greengard and V Rokhlin. A fast algorithm for particle simulations. *Journal of Computational Physics*, 73(2):325–348, 1987.
- [4] P.G. Martinsson. *Fast Direct Solvers for Elliptic PDEs*. CBMS regional conference. Society for Industrial and Applied Mathematics, 2020.
- [5] Bo Zhang, Jingfang Huang, Nikos P. Pitsianis, and Xiaobai Sun. Recfmm: Recursive parallelization of the adaptive fast multipole method for coulomb and screened coulomb interactions. *Communications in Computational Physics*, 20(2):534–550, 2016.